

DataLab 实验报告

一、实验基本信息

1.1 实验名称: DataLab 位运算与浮点数表示实验

1.2 姓名: 严泳垚

1.3 学号: 24307090048

1.4 完成日期: 2026 年 10 月 9 日

二、实验运行结果

2.1 运算符规则检查结果 (执行 `./check_ops.py bits.c` 后的截图。)

```
● @Yyyilanli → /workspaces/DataLab (main) $ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 11/16 operators
roundEvenPow2: 24/24 operators
midpointTowardFirst: 20/32 operators
isBetweenEitherOrder: 47/48 operators
mul5Sat: 15/30 operators
classifyAdd9: 27/52 operators
floatScaleThreeHalves: 37/60 operators
floatRoundEven: 29/65 operators
float_i2f: 34/40 operators
bitCount: 34/40 operators
bitReverse: 32/34 operators
All 19 functions passed operator checks.
○ @Yyyilanli → /workspaces/DataLab (main) $
```

2.2 正确性测试结果 (执行 `./btest` 后的截图)

```

@Yyyilanli → /workspaces/DataLab (main) $ ./btest
Score  Rating  Errors  Function
1      1      0      signMask
2      2      0      bitXor
3      3      0      negativePart
4      4      0      copyByteWithin
4      4      0      logicalShift
4      4      0      swapNibblePairs
4      4      0      secondLowestZeroBit
5      5      0      oddParity
5      5      0      rotateRightBits
5      5      0      roundEvenPow2
5      5      0      midpointTowardFirst
7      7      0      isBetweenEitherOrder
7      7      0      mul5Sat
7      7      0      classifyAdd3
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 110/110

```

三、各函数实现思路

3.1 基础整数位运算（P1-P9）

1. `signMask(x)`: 利用 C 语言有符号数算术右移补符号位的特性，把 `x` 右移 31 位：正数右移补 0 得到全 0 掩码，负数右移补 1 得到全 1 掩码，直接返回结果。

2. `bitXor(x, y)`: 题目禁止直接用 `^` 运算符，根据异或逻辑等价公式 $x \oplus y = (\sim x \& y) \mid (x \& \sim y)$ ，用与、或、非三个基础运算组合实现异或功能。

3. `negativePart(x)`: 要返回 `x` 中的负数部分（正数返回 0，负数返回原值），直接让 `x` 和它符号扩展出来的掩码按位与：正数和 0 相与得 0，负数和全 1 相与保留原值。

4. `copyByteWithin(x, n)`: 先把 `x` 右移 `n*8` 位，把目标字节移到最低 8 位，和 `0xFF` 掩码按位与提取出这个字节；再把提取出的字节左移 `k*8` 位到目标位置，和 `x` 原其他部分拼接完成复制。

5. `logicalShift(x, n)`: C 语言默认右移是算术右移会补符号位，实现逻辑右移时，先正常右移 `n` 位，再用低 $(32-n)$ 位全 1 的掩码把高位补出来的符号位全部清 0，得到无符号右移结果。

6. `swapNibblePairs(x)`: 要交换每个字节内部的高低 4 位（半字节），用 `0x0F` 掩码提取低 4 位左移 4 位，用 `0xF0` 掩码提取高 4 位右移 4 位，两部分按位或完成交换。

7. `secondLowestZeroBit(x)`: 从低位开始找第二个为 0 的位, 先把所有为 1 的位通过运算标记, 逐级向右筛选, 定位到第二个 0 的位置, 返回对应为 1 的位掩码。

8. `oddParity(x)`: 统计 x 里二进制 1 的个数奇偶性, 用分治累加的方式, 逐步把相邻位的 1 个数汇总, 最后看最低位是 0 还是 1, 得到奇偶校验结果。

9. `rotateRightBits(x, n)`: 实现循环右移, 右移 n 位丢失的低 n 位不能扔: 把低 n 位移到高 n 位位置, 和右移后的高部分按位或起来, 实现首尾相接的循环移位。

3.2 溢出与区间判断 (P10-P14)

10. `roundEvenPow2(x, n)`: 把非负 x 舍入到最近的 2^n 倍数, 正好在中间时舍入到偶数倍数。先算出低 n 位的余数, 判断余数和半值 $2^{(n-1)}$ 的大小, 正好中间时看舍入后结果奇偶性决定方向, 最后把低 n 位清 0, 需要进位就加上 2^n 。

11. `midpointTowardFirst(a, b)`: 计算 a 和 b 的中点, 直接 $a+b$ 会溢出, 拆成两部分算: 高位部分直接平均, 最低位单独进位补值, 保证运算过程不溢出, 结果向第一个输入 a 靠近。

12. `isBetweenEitherOrder(x, a, b)`: 判断 x 是否在 a 、 b 构成的闭区间内 (端点顺序不限), 通过有符号数减法的符号位, 分别比较 x 和 a 、 x 和 b 的大小, 覆盖 $a \leq x \leq b$ 和 $b \leq x \leq a$ 两种区间顺序。

13. `mul5Sat(x)`: 计算 $x*5$, 溢出时饱和到 `int` 最大/最小值。先通过 $x \ll 2 + x$ 算出初步结果, 再通过结果和原 x 的符号位变化判断是否溢出, 正溢出返回最大正数, 负溢出返回最小负数。

14. `classifyAdd3(x)`: 判断 $x+3$ 是正常、上溢还是下溢。先算 $x+3$ 的结果, 通过符号位前后变化判断溢出方向, 输出对应三类结果的标记值。

3.3 浮点数运算 (P15-P17)

15. `floatScaleThreeHalves(uf)`: 把输入浮点数乘以 1.5。先拆符号位、阶码、尾数三部分, 先处理 0、无穷大、NaN 这些特殊值直接返回; 规格化数阶码加 1 调整, 非规格化数单独按尾数运算规则处理。

16. `floatRoundEven(val)`: 按 IEEE754 就近舍入到偶数规则, 先看要舍去的尾数部分: 大于半值就进位, 小于半值就截断, 正好在半值时看保留的尾数位是奇数

还是偶数，偶数截断、奇数进位。

17. `float_i2f(n)`: 把整数转成单精度浮点数。先记录符号位，找到整数最高位的位置计算阶码，把整数移位对齐到尾数位置，处理移位不够的情况，按舍入规则补尾数，最后拼接符号、阶码、尾数三部分。

3.4 分治位运算 (P18-P19)

18. `bitCount(x)`: 统计 32 位二进制里 1 的总个数，采用分治并行累加：先相邻两位求和，再相邻四位求和、八位求和，最后所有位的 1 数累加，全程只用位运算不循环。

19. `bitReverse(x)`: 把 32 位的二进制顺序完全翻转。同样用分治思想：先交换相邻 1 位，再交换相邻 2 位、4 位、8 位、16 位，经过 5 次分组交换，就完成全位翻转。

四、参考资料

4.1 Randal E. Bryant, David R. O'Hallaron. 深入理解计算机系统（第 3 版）第 2 章：信息的表示与处理。机械工业出版社.

4.2 IEEE Computer Society. IEEE Standard for Floating-Point Arithmetic (IEEE 754-2008).

4.3 Carnegie Mellon University. CSAPP: Data Lab 官方实验文档.

4.4 课堂课件与实验指导书: Lab1 DataLab 实验说明.

4.5 位运算分治技巧、补码溢出检测相关公开技术博客与 CSAPP 习题解析资料.

五、实验总结与感受

通过这次 DataLab 实验，我从最开始只会照着例题写位运算，到最后能独立处理溢出判断、浮点数舍入这些细节，对计算机底层二进制表示的理解比上课听讲要深刻很多。最开始我以为只要代码跑通测试就行，后来才发现还有运算符数量限制、不能用大常量、有符号右移会补 1 这些容易踩的坑——比如 `bitReverse` 一开始怎么都通不过 `0x80000000` 的测试，查了很久才反应过来是算术右移带进来了多余的符号位。尤其是分治实现 `bitCount` 和 `bitReverse` 那部分，

从 1 位相邻交换逐步扩展到 16 位分组处理的思路，让我第一次感受到位运算虽然看起来零散，但其实可以用很优雅的分治方法解决复杂问题。整个调试过程虽然反复改了很多次，但最后拿到 110/110 满分的时候收获感很强，也算是真正把补码、IEEE754 这些课本上的知识点落地到了代码里。